Automated Digital Circuit Tester

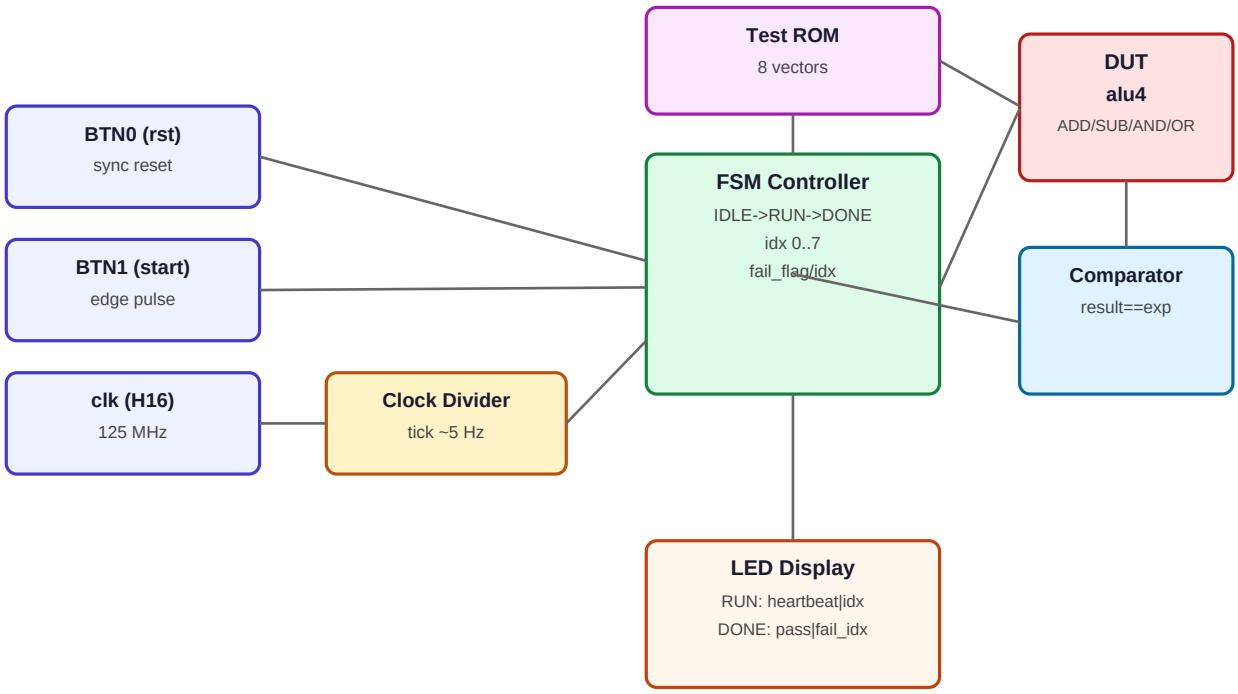
V-SPACE FPGA Build Challenge 2026 Team: Mahasena | **Team Lead:** Manaswi | **Experiment 1 — Beginner Board:** PYNQ-Z2 (Xilinx Zynq XC7Z020CLG400-1) | **Tool:** AMD Vivado 2026.1

1. Objective

Manually testing a digital circuit by hand-toggling inputs and reading outputs does not scale and is error-prone. This experiment builds a small, self-contained **Automated Test Equipment (ATE)** block in hardware: a finite-state-machine (FSM) controller that automatically drives a bank of pre-defined test vectors into a Device Under Test (DUT), compares every result against the expected answer, and reports a clear **PASS** or **FAIL (+ failing vector index)** on the board LEDs — with no human intervention once the START button is pressed.

The DUT chosen for this demonstration is a 4-bit ALU supporting **ADD, SUB, AND, OR**, which is complex enough to have genuinely interesting corner cases (carry/borrow, zero operands) while remaining small enough to verify completely by hand.

2. Block Diagram



See Images/block_diagram.svg.

The system has five blocks:

1. **Clock Divider** — derives a slow, human-visible `tick` from the 125 MHz board clock.
2. **Test Vector ROM** — an 8-entry lookup table (`a`, `b`, `opcode`, expected `cout`, expected `result`), implemented as a Verilog `function`/`case` (synthesizes to a small ROM/MUX).
3. **FSM Controller** — sequences through the ROM (`IDLE -> RUN -> DONE`), applies each vector, and latches the first mismatch.
4. **DUT (`alu4`)** — the 4-bit ALU under test, purely combinational.
5. **LED Display** — encodes live progress and the final verdict.

---

3. RTL Design

3.1 Module hierarchy

```
top_module                (Automated Digital Circuit Tester, FSM + ROM + display)
  +-- alu4                (Device Under Test — 4-bit ALU)
```

3.2 Verilog code structure

`alu4` — combinational 4-bit ALU. Each operation is computed with explicit zero-extension before addition (`{1'b0,a} + {1'b0,b}`) so the carry-out bit is always correctly and unambiguously derived, avoiding any Verilog width-inference ambiguity:

```
always @(*) begin
    case (opcode)
        2'b00:   {cout, result} = {1'b0, a} + {1'b0, b};            // ADD
        2'b01:   {cout, result} = {1'b0, a} + {1'b0, ~b} + 5'b00001; // SUB
        2'b10:   {cout, result} = {1'b0, (a & b)};                  // AND
        2'b11:   {cout, result} = {1'b0, (a | b)};                  // OR
        default: {cout, result} = 5'b00000;
    endcase
end
```

For SUB, `cout = 1` means **no borrow** (a >= b); `cout = 0` means a borrow occurred.

`top_module` — the FSM walks `idx` from 0 to 7, one step per `tick`:

```
S_RUN: begin
    if (tick) begin
        if (!vector_pass && !fail_flag) begin
            fail_flag <= 1'b1;
            fail_idx  <= idx;
        end
        if (idx == 3'd7) state <= S_DONE;
        else             idx  <= idx + 1'b1;
    end
end
```

Full source: RTL/top_module.v, constraints: RTL/constraints.xdc.

3.3 LED encoding

State	LED[3]	LED[2:0]
RUN	tick heartbeat	current vector index
DONE	1 = PASS, 0 = FAIL	first-fail index (000 if PASS)
IDLE	0	000

---

4. Simulation Results

Simulated with the **Vivado Simulator (xvlog / xelab / xsim)**, standalone CLI flow. Testbench: `Testbench/tb_top.v`. Raw transcript: `Simulation/transcript.txt`. Waveform database: `Simulation/tb_top_sim.wdb` (open in Vivado -> Open Waveform Database for a GUI screenshot).

Test strategy (3 phases):

1. An independent, hand-written **golden reference model** cross-checks every expected ROM value (catches ROM *and* ALU bugs).
2. The real **hardware FSM** is run end-to-end through all 8 vectors via the `start` pulse.
3. **Reset** is asserted mid-run to confirm the FSM returns cleanly to `IDLE`.

Actual transcript output (2026-10-10):

```
================================================
 Automated Digital Circuit Tester - Testbench
================================================

-- Phase 1: Golden-model cross-check --
  [OK]   vector 0: a=3 b=2 op=00 -> cout=0 result=5
  [OK]   vector 1: a=f b=1 op=00 -> cout=1 result=0
  [OK]   vector 2: a=5 b=3 op=01 -> cout=1 result=2
  [OK]   vector 3: a=2 b=5 op=01 -> cout=0 result=d
  [OK]   vector 4: a=a b=6 op=10 -> cout=0 result=2
  [OK]   vector 5: a=c b=3 op=11 -> cout=0 result=f
  [OK]   vector 6: a=0 b=0 op=00 -> cout=0 result=0
  [OK]   vector 7: a=9 b=9 op=01 -> cout=1 result=0

-- Phase 2: Hardware FSM test sequence --
  Final LED = 1000 (led[3]=PASS/FAIL, led[2:0]=index)
  [PASS] All 8 vectors passed, as expected for a correct ALU.

-- Phase 3: Reset behaviour --
  [PASS] Reset clears LEDs and returns FSM to IDLE.

================================================
 TESTBENCH SUMMARY: ALL CHECKS PASSED (0 errors)
================================================
```

■ TO DO:
Manaswi: open `Simulation/tb_top_sim.wdb` in the Vivado GUI waveform viewer, take a screenshot of the `idx`, `dut_result`, `fail_flag`, `led` signals over time, and save it as `Images/waveform.png`.

5. Hardware Implementation

■ **TO DO:**
pending: synthesis/implementation bitstream generation is blocked by a Vivado license-tier issue (the installed license defaults to the ALVEO tier, which does not cover the Zynq-7000 / XC7Z020 part). The same license file also contains a WebPACK/BASIC tier that **does** cover XC7Z020 — switch to it via **Vivado -> Help -> Manage License**, then re-run:
`vivado -mode batch -source run_vivado_full.tcl`
from the repository root. Once the `.bit` file is generated it will appear at `Bitstream/top_module.bit` and should be programmed onto the PYNQ-Z2 for the final hardware screenshots/photos below.

■ **TO DO:**
Manaswi: after programming the board, add:
`Images/board_setup.jpg` — photo of the PYNQ-Z2 wired/powered and ready
`Images/hardware_output.jpg` — photo of the LEDs showing a PASS result (`1000`)
Resource utilization report (optional): `Simulation/impl_utilization.rpt`

6. Applications

- **Manufacturing test**: the same architecture (ROM of vectors + comparator + FSM) is exactly how real Automated Test Equipment (ATE) screens chips coming off a production line.
- **Regression testing for IP cores**: every time an RTL block changes, re-running a fixed vector bank catches regressions automatically, without a human re-checking waveforms.
- **Built-In Self-Test (BIST)**: the same pattern, scaled up, is used inside real SoCs to self-check sub-blocks at power-up.
- **Educational tool**: demonstrates FSM design, ROM-based sequencing, and self-checking verification in one compact project.

7. Conclusion

This experiment implemented a working, fully-simulated automated test sequencer for a 4-bit ALU, verified with an independent golden model and a cycle-accurate hardware FSM simulation — both reporting **zero errors**. The design cleanly separates the **DUT** (`alu4`), the **test infrastructure** (ROM + FSM + comparator), and the **display** layer, making it straightforward to swap in a different/larger DUT for future extension.

Future scope: this tester currently stops reporting after the *first* failing vector; a natural extension is to accumulate and report a *total fail count*, or to make the DUT swappable at build time so the same tester can validate multiple circuit variants automatically — a direction this team plans to explore further in the

FAULTX – HIL Fault Injection experiment later in this challenge.

Document prepared for the V-SPACE FPGA Build Challenge 2026 — Team Mahasena.